Readme: Speech Emotion Recognition

MSI5001 2026/2027 Semester 1 | Team 12
Chow Lip Khin, Manish Agarwal, Tan Sumengyuan, Xu Jin

Reproduce baseline and optimised results using **RUN_MODE = 'load'**, the default. No training or hyperparameter searches are required. The original dataset is still needed for feature extraction and saved-model checks.

1. Extract the code and download the external files

Extract **MSI5001_Team12_Code.zip** into a writable folder. It contains **MSI5001_Team12_Code.ipynb**, **Readme.pdf** and **links.txt**. Open links.txt and download the saved-model archives for both basic and rich. Extract them beside the notebook, retaining all files and subfolders.

The **dataset link is optional**: reuse your original AESDD copy if available, or download it from the dataset link when provided. Extract the audio under AESDD. Before running, check that the working folder has this layout:

Working folder after downloads	Purpose
MSI5001_Team12_Code.ipynb	Full implementation; sections 1-11.
Readme.pdf	These instructions.
links.txt	Saved-model links; optional dataset link.
AESDD/	Existing copy or optional download.
saved_models/basic/20261019T061934Z_ecaaf52a/	Basic archive after extraction.
saved_models/rich/20261019T073027Z_31bb74b0/	Rich archive after extraction.

Keep manifest.json, evaluation_state.joblib, model weights, fitted scalers and CNN details together in each run folder. If a downloaded ZIP has an extra parent folder, move its contents so the paths above match. The fixed run IDs do not require latest.json.

2. Match the recorded environment

Both run manifests record these versions. Use Python 3.14, preferably the recorded patch release, and match the serialisation-related packages before loading. CPU execution is supported; a GPU is optional.

Component	Recorded version	Component	Recorded version
Python	3.14.7	PyTorch	2.14.1+cu130
NumPy	2.5.4	SciPy	1.18.1
pandas	3.0.6	scikit-learn	1.9.1
librosa	1.0.0	soundfile	0.14.0
joblib	1.6.0	Other imports	matplotlib, gdown, IPython

The final three libraries are imported but their versions are not recorded in the manifests. Locally, use JupyterLab or VS Code with the Python/Jupyter extensions.

Set up the notebook environment

3. Local Windows setup

Install Python 3.14. Open the extracted project folder in VS Code and open a PowerShell terminal in that folder. Run these commands in order; environment activation is not required.

```
py -3.14 -m venv .venv
$SER_PY = ".\.venv\Scripts\python.exe"
& $SER_PY -m pip install --upgrade pip
& $SER_PY -m pip install ipykernel jupyterlab
& $SER_PY -m ipykernel install --user --name msi5001-ser
```

Open **MSI5001_Team12_Code.ipynb** and select the **msi5001-ser** kernel, or select the Python executable in .venv. On Linux/macOS, create the environment with `python3.14 -m venv .venv` and use `.venv/bin/python` for the same setup commands.

4. Install dependencies once, then restart

Run the following in a temporary notebook setup cell using the selected kernel. Skip the notebook's original unpinned `%pip install --upgrade ...` cell during saved-run reproduction.

```
%pip install numpy==2.5.4 scipy==1.18.1 pandas==3.0.6
%pip install scikit-learn==1.9.1 librosa==1.0.0 soundfile==0.14.0
%pip install joblib==1.6.0 torch==2.14.1
%pip install matplotlib gdown ipython
```

Restart the kernel after installation, then run the import cell in section 1. The recorded PyTorch build is 2.14.1+cu130. For local NVIDIA training, obtain a matching CUDA-enabled build using the official PyTorch installation instructions. Keep the trained-model package version aligned. If a recorded version has no wheel for your platform, use a compatible platform/original environment rather than silently changing model-library versions.

Alternative: Google Colab

Upload MSI5001_Team12_Code.ipynb to Colab. In a new setup cell before section 1, upload the implementation ZIP and extract it into a separate folder:

```
from google.colab import files
files.upload()  # Choose MSI5001_Team12_Code.zip
import os, zipfile
with zipfile.ZipFile('/content/MSI5001_Team12_Code.zip') as z:
    z.extractall('/content/ser_project')
os.chdir('/content/ser_project')
```

Next, open **links.txt** in the extracted project folder. Download the saved-model ZIP archives and, if needed, the dataset ZIP. In a new setup cell, upload those archives and extract them:

```
uploaded = files.upload() # Model ZIPs; dataset ZIP if needed
for name in uploaded:
    with zipfile.ZipFile(name) as z:
        z.extractall('/content/ser_project')
```

Check that AESDD and saved_models sit directly under /content/ser_project, as shown on page 1. Install dependencies, restart if prompted, and restore this working directory afterward. Colab's Python version is runtime-dependent; if the recorded versions cannot be installed, use the local environment. GPU selection is optional for load mode. Colab storage is temporary.

Reproduce basic and rich results

5. Set the saved-run configuration before running all cells

RUN_MODE defaults to 'load'. After imports, check the Run configuration cell in section 1 and select the matching feature preset and saved-run ID. To reproduce basic, use:

```
FEATURE_SET = 'basic'
RUN_MODE = 'load'
SAVE_MODELS = True
MODEL_ROOT = Path('./saved_models')
MODEL_RUN = '20261019T061934Z_ecaaf52a'
```

To reproduce **rich**, use the same settings except for:

```
FEATURE_SET = 'rich'
MODEL_RUN = '20261019T073027Z_31bb74b0'
```

SAVE_MODELS is ignored in load mode. Use the explicit run IDs rather than latest so the chosen archives remain fixed.

6. Point section 2 to the original dataset

```
SOURCE = 'local'
LOCAL_PATH = r'./AESDD'
```

Use your existing AESDD copy, or download it using the optional dataset link in links.txt and extract it to ./AESDD. Section 3 expects 605 candidates and excludes s05 (3).wav, leaving 604 usable recordings. Keep WAV filenames and bytes unchanged; section 4 handles preprocessing. Alternatively, set SOURCE = 'drive' and DRIVE_URL to the public dataset URL from links.txt. The notebook copies the data locally without mounting Drive.

7. Run sections 1-11 in order

Sections	What happens in load mode
1-4	Import libraries, choose run, load/clean audio and extract features.
5-6	Build outer splits; verify archived inputs and restore scaling.
7-8	Restore baseline models and produce baseline evaluation.
9-10	Restore selected settings/models and regenerate final comparisons.
11	Reload every selected model; verify predictions and score tables.

Skip only the installation cell once dependencies are ready. Execute the configuration cell before sections 2-11. Load mode still reads audio, extracts features and performs inference; it should not fit models or run searches.

8. Run the other feature preset

Save the first run's notebook output under a distinct local filename. Restart the kernel, select the other FEATURE_SET and its matching MODEL_RUN, and rerun the imports and sections 1-11 with the installation cell skipped. A clean restart keeps the two evaluations separate.

Verify the results and resolve errors

9. Expected evaluation and saved-model checks

Each preset evaluates seven models: DummyClassifier, KNN, Logistic Regression, Random Forest, SVM, MLP and CNN. Both stages share **5 random stratified group folds** and **6 LOSO speaker holdouts**. Each recording receives one test prediction per protocol. Each archive contains **154 selected models** (7 models x 11 folds x 2 stages).

Preset / selected optimised model	Random accuracy	Random macro F1	LOSO accuracy	LOSO macro F1
Basic / SVM	74.2%	0.741	45.2%	0.441
Basic / CNN	68.5%	0.678	45.4%	0.442
Rich / Logistic Regression	71.7%	0.713	55.5%	0.542
Rich / CNN	68.0%	0.672	57.3%	0.573

These are pooled out-of-fold values, rounded for display; they are not means of speaker F1 scores. Complete baseline/optimised results appear in sections 8 and 10. Basic vectors have 40 MFCC summary values and CNN input 1 x 40 x 128; rich vectors have 120 values and CNN log-mel input 1 x 64 x 128.

Section 11 should report matching reloaded predictions and pass the score-table assertions. Archive checks also compare file hashes, recording identities, class labels, preprocessing and outer splits. Loaded neural models use CPU inference. Fresh retraining can produce different neural results even with the same split seed.

Troubleshooting

Problem	Action
Missing package or wrong kernel	Install in the selected notebook kernel; restart it. Check sys.executable.
Saved run/file not found	Download saved models from links.txt; check MODEL_ROOT, run ID and extracted folders.
Dataset or preprocessing mismatch	Restore the pinned notebook and original WAVs; match the feature preset. Keep the built-in checks enabled.
Pickle/version error or version warning	Match the manifest versions, especially pandas, scikit-learn, NumPy and joblib.
Reloaded predictions differ	Confirm code, dataset and versions match. Retain the error/output for diagnosis; do not bypass assertions.

Optional: train new models

Use RUN_MODE = 'train' and SAVE_MODELS = True, select the preset, then run sections 1-11. This runs all baselines and nested tuning and can take substantially longer. New runs are saved under a new timestamped folder; keep the two fixed submission archives. GPU support applies to PyTorch training, while classical searches use CPU.

Implementation source

GitHub: lipkhin/msi5001-ser, snapshot f449123. Notebook: MSI5001_Team12_Code.ipynb. Run IDs and package versions above come from the complete basic/rich saved-model manifests. Retain Readme.pdf beside the notebook in MSI5001_Team12_Code.zip.